

Prague University of Economics and Business

Faculty of Informatics and Statistics



DATA STREAMING ARCHITECTURE

BACHELOR THESIS

Study programme: Applied Informatics

Field of study: Information Technologies

Author: Ilia Maksimenko

Supervisor: Ing. et Ing. Soňa Karkošková, Ph.D.

Prague, May 2023

Declaration

I hereby declare that I am the sole author of the thesis entitled “Data Streaming Architecture”. I duly marked out all quotations. The used literature and sources are stated in the attached list of references.

In Prague on

.....

Ilia Maksimenko

Abstract

The proliferation of big data and the emergence of real-time applications have created a need for scalable and efficient data processing systems that can handle large volumes of data in real-time. The development of the Internet of Things is taking this to another level altogether; now data can come to us in real time, and our goal, in turn, will be to transfer it without loss, in the right order, and at the highest possible speed. In this thesis, author proposes a general Data Streaming Architecture. The theoretical part focuses on outlining the fundamental ideas required to comprehend the topics addressed in the thesis.

The primary part of this bachelor's thesis delves into the research surrounding Data Streaming Architecture through a Systematic Literature Review. The author begins by choosing relevant papers and subsequently analyzes them to emphasize the components of the proposed architecture present in each examined paper. Using scientific methods, such as generalization and synthesis, the author identifies common components in the architectures reviewed.

Finally, based on the identified components, the author puts forth a proposal of a general Data Streaming Architecture, with each component of the architecture being commented for a more in-depth overview.

Keywords

Data streaming architecture, stream processing, Systematic Literature Review, data stream

Table of Contents

Introduction	6
<i>Motivation</i>	<i>6</i>
<i>Definition of the Topic</i>	<i>7</i>
<i>Expected Contribution</i>	<i>7</i>
<i>Aim of the Thesis.....</i>	<i>7</i>
<i>Methods of Scientific Work.....</i>	<i>7</i>
1 Data Streaming	9
1.1 <i>Characteristics of Data Streams</i>	<i>9</i>
1.2 <i>Components of Data Streaming Systems</i>	<i>9</i>
1.3 <i>Challenges Associated with Processing Data Streams.....</i>	<i>10</i>
2 Literature Review	12
<i>Research Objectives</i>	<i>12</i>
<i>Research Questions.....</i>	<i>12</i>
2.1 <i>Evolution of Data Streaming Architectures</i>	<i>13</i>
2.1.1 <i>Early Stages: Batch Processing and Simple Event Processing</i>	<i>13</i>
2.1.2 <i>Emergence of Complex Event Processing (CEP) Systems</i>	<i>13</i>
2.1.3 <i>Distributed Stream Processing Systems</i>	<i>13</i>
2.1.4 <i>Unified Stream and Batch Processing</i>	<i>13</i>
2.1.5 <i>Modern Data Streaming Architectures</i>	<i>14</i>
2.2 <i>Methodology of Systematic Literature Review.....</i>	<i>14</i>
2.2.1 <i>Search Strategy</i>	<i>14</i>
2.2.2 <i>Study Selection Criteria.....</i>	<i>15</i>
2.2.3 <i>Data Extraction</i>	<i>15</i>
2.2.4 <i>Meta Analysis Search</i>	<i>15</i>
2.2.5 <i>Scientific Papers Search</i>	<i>16</i>
<i>Analysis.....</i>	<i>17</i>
2.3 <i>Architectures Identification.....</i>	<i>18</i>
2.3.1 <i>Kafka Usage at LinkedIn Architecture.....</i>	<i>18</i>
2.3.2 <i>Concept of Global Design for Streaming Architecture</i>	<i>19</i>
2.3.3 <i>Streaming Data Architectural Blueprint</i>	<i>21</i>
2.3.4 <i>ETL Streaming Architecture</i>	<i>22</i>
2.3.5 <i>Architecture For Decision Support.....</i>	<i>24</i>
2.3.6 <i>General Architecture of a DSPS.....</i>	<i>25</i>
2.3.7 <i>Upsolver Reference Architecture.....</i>	<i>27</i>
2.4 <i>Synthesis of Described Components.....</i>	<i>28</i>
2.5 <i>Occurrences of Components in Architectures</i>	<i>30</i>
2.6 <i>Discussion</i>	<i>31</i>
2.6.1 <i>Research Questions of the Systematic Literature Review.....</i>	<i>32</i>
2.6.2 <i>Limits of Research Conducted</i>	<i>32</i>

3 General Data Streaming Architecture Proposal34

3.1 Data Source Layer34

3.2 Collection Layer35

3.3 Stream Processing Layer.....35

3.4 Output Layer.....35

3.5 In-Memory Storage Layer.....36

3.6 Long-Term Storage Layer36

3.7 Analytics Layer36

3.8 Data Quality Layer37

Conclusion.....38

Suggestions for Future Research.....39

Sources.....40

Introduction

The exponential growth of data has created a new era in computing where the focus is shifting towards real-time processing and analysis of large-scale data. The sheer volume, velocity, and variety of data that are being generated from various sources have made it necessary to design efficient and scalable data processing systems that can handle the data streams in real-time.

In this rapidly evolving field, it is critical to build the right architecture to make the most effective use of new technologies. The more technology evolves, the more important it is to choose the right architecture components and the connections between them. The reason for this is that the companies are increasingly dependent on technology, and in some areas where it is critical to use data streaming, the wrong architecture can cause problems, such as problems with data access or even data loss. Which, in turn, with the development of approaches such as Data-Driven Decision-Making (DDDM) has a huge impact on decision-making and the consequent loss of company finances or other assets.

The aim of this thesis is to propose a general data streaming architecture. This thesis begins with an overview of the current state-of-the-art in data streaming and its challenges, including data ingestion, processing, storage, and retrieval. The author then reviews relevant works in the field and tries to identify common components which are then to be used in final proposal.

The research presented in this thesis contributes to the ongoing efforts to propose efficient general Data Streaming Architecture. Since the suggested architecture is only a general proposal, this paper does not deal with the implementation of the architectures reviewed or proposed. The work considers the architectures at the level of their components highlighted by the authors of the original works.

Motivation

The motivation for this thesis stems from the increasing demand for real-time processing and analysis of large-scale data streams. Real-time processing of data streams presents a unique set of challenges and opportunities that are not typically encountered in batch processing, such as handling large volumes of data, processing data in real-time, ensuring data consistency, fault tolerance, and scalability.

To effectively solve the above problems, it is necessary to have an idea of what the architecture that will provide the solution to them should look like.

Ultimately, this thesis seeks to inspire further research and innovation in the field of real-time data streaming.

Definition of the Topic

The bachelor thesis first defines the concepts of data stream and data streaming system. These concepts are then considered together, in the context of the concept of Data Streaming Architecture, its components and challenges. After analyzing the different approaches to Data Streaming Architecture and generalizing them, the author forms a general representation of the architecture using scientific methods defined in the “Methods of Scientific Work” section.

Expected Contribution

As the main contribution of this thesis the author considers the proposal of a general Data Streaming Architecture derived using scientific methods. Since most of the current architecture proposals are very much related to the specific implementation and the specific frameworks used during the implementation, the author considers it important to have a general idea of it.

Aim of the Thesis

The aim of this bachelor thesis is to create a proposal for a general Data Streaming Architecture based on a review of already existing architectures.

Achievement of this aim will be carried out with the help of following objectives:

1. Analyze scientific literature and papers on Data Streaming Architecture using Systematic Literature Review.
2. Derive commonalities among the analyzed works and summarize them in the final general architecture proposal.

Methods of Scientific Work

In this bachelor thesis, several methods of scientific work will be employed to ensure a comprehensive and rigorous exploration of Data Streaming Architectures. The methods to be used include:

1. Systematic Literature Review: a systematic review of existing literature, including research papers, technical reports, and books, will be conducted to understand the current state of knowledge in data streaming architectures. It is a form of secondary study as defined by the authors of methodology (Kitchenham & Charters, 2007).
2. Analysis: the purpose of the method is to determine the parts of the whole. In this paper, the method is used to study relevant architectures and identify their components.

3. Induction: in this paper is understood as “inductive reasoning”. The author will make various statements, but they will only relate to the works reviewed and can by no means be axiomatic. This is the result of the fact that the method itself only works with the statements based on the considered information and does not exclude the possibility of the existence of another information that does not correspond to the statements made.
4. Generalization: generalization is an essential tool in the scientific method that allows researchers to draw broader conclusions and make predictions based on specific observations or data. In this thesis, the method is used to draw broader inferences about streaming architectures from a limited pool of reviewed data.
5. Synthesis: a method that involves combining parts into a whole. In this work, it will be used at the very end to create a proposal of general Data Streaming Architecture, based on the components identified in the previous steps.

1 Data Streaming

Data streaming refers to the continuous flow of data, generated by various sources, and processed in real-time or near-real-time to derive insights and support decision-making processes. With the rapid growth of data from social media, IoT devices, and online transactions, the demand for data streaming has increased significantly. This chapter discusses the fundamentals, components, and the challenges associated with processing data streams.

1.1 Characteristics of Data Streams

Data streams are continuous and dynamic sequences of data generated by various sources, such as social media, IoT devices, and online transactions. They have distinct characteristics compared to traditional static datasets (Babcock et al., 2002) which necessitate specialized approaches for processing and analysis. These characteristics include:

- **Infinite and Unbounded:** unlike static datasets, data streams are potentially infinite and unbounded, as they continuously flow with no predetermined end (Golab & Özsu, 2003). This characteristic necessitates the development of streaming architectures that can efficiently process data in a continuous and incremental manner, without relying on traditional batch processing methods.
- **Infinite and Continuous:** data streams are unbounded, with new data continuously generated and arriving at varying rates (Cugola & Margara, 2012).
- **Temporal:** data in streams have inherent time-based properties, such as timestamps or time windows, which influence processing and analytics (Babcock et al., 2002).
- **Heterogeneous and Variable:** data streams can be heterogeneous and variable in terms of data types, formats, and quality (Zhu & Shasha, 2002). This variability poses challenges in data integration, cleansing, and processing, requiring flexible and adaptable data streaming architectures to handle diverse data sources.
- **Real-time or near-real-time processing:** data streams demand low-latency processing to deliver timely insights and support decision-making processes (Cugola & Margara, 2012).

1.2 Components of Data Streaming Systems

Data streaming systems typically comprise three main components (Cugola & Margara, 2012):

- **Data producers:** data producers are the sources that generate continuous streams of data to be ingested and processed by the data streaming system. These can include sensors, web applications, social media platforms, or any other source that produces

data in real-time. Data producers often emit data in various formats, and it is essential for data streaming systems to be able to accommodate this diversity.

- The stream processing engine: is responsible for processing and analyzing the ingested data streams in real-time. This component performs various operations on the data, such as filtering, aggregation, transformation, and enrichment, according to the requirements of the specific use case (Affetti et al., 2017). The processing engine must be capable of handling high throughput and providing low-latency results while ensuring fault tolerance and scalability. Examples of data processors include Apache Kafka, Apache Flink, and Apache Pulsar.
- Data consumers or Data sinks: are the destinations where the processed data is stored or output for further consumption. These can include databases, file systems, data warehouses, or other systems that serve as a repository for the results of the data streaming system's processing. Data sinks must be able to handle the high throughput of the processed data streams and provide low-latency access for downstream applications or analytics.

Although monitoring and alerting are not the main components, they are still worth mentioning. Monitoring and alerting components are essential for maintaining the health and performance of data streaming systems. These components track various metrics, such as data processing latency, throughput, resource utilization, and error rates, enabling system administrators to identify potential issues and take corrective action promptly. Monitoring and alerting tools can include built-in features of the data streaming system or external solutions like Prometheus, Grafana, and Elasticsearch.

1.3 Challenges Associated with Processing Data Streams

Data stream processing has emerged as a crucial component in modern data processing architectures, as organizations increasingly demand real-time analytics and insights from the massive volumes of data generated by various sources.

For example, as (Dunning & Friedman, 2016) point: "... partnerships between British Railways, Cisco Systems, and telecommunication companies are building connected systems to reduce risk for British trains. Heavily equipped with sensors, the trains monitor the tracks, and the tracks monitor the trains while also communicating with operating centers. Data such as information about train speed, location, and function as well as track conditions are transmitted as continuous streams of data that make it possible for computer applications to provide low-latency insights as events happen. In this way, engineers are able to take action in a timely manner."

However, processing data streams efficiently and effectively presents several challenges.

One of the primary challenges in processing data streams is managing the high volume and velocity of data. As data is generated continuously and at a rapid pace, streaming systems must be able to process, store, and analyze the data without causing bottlenecks or delays (Cugola & Margara, 2012).

Another challenge would be low latency. Real-time analytics necessitates low-latency processing to deliver timely insights and enable rapid decision-making. Data streaming architectures should be designed to minimize the time it takes to ingest, process, and analyze data, thereby providing near-instantaneous results.

Scalability is a fundamental requirement for data streaming architectures as they must be able to handle varying workloads and process large-scale data streams efficiently. Data stream processing systems must be able to scale up or down according to the varying data loads, ensuring consistent performance and resource efficiency. Horizontal scaling, which involves adding more machines to a system, is a common approach to achieving scalability. This requires effective load balancing, partitioning, and distribution mechanisms to handle fluctuating workloads (Toshniwal et al., 2014).

Fault tolerance is essential in data streaming architectures to ensure that data processing can continue without interruption despite potential failures in system components. Ensuring fault tolerance and reliability in data stream processing systems is crucial to prevent data loss and maintain system availability. This requires mechanisms for detecting failures, handling replication, and facilitating recovery while minimizing the impact on system performance.

Maintaining consistency and managing state in data stream processing systems can be challenging, especially in distributed environments with multiple nodes. Ensuring data consistency requires effective coordination between nodes and the ability to manage stateful computations (Carbone et al., 2015).

Data streams often contain noisy, incomplete, or inconsistent data, which can negatively impact the accuracy of analytics and insights derived from the processed data. Dealing with data quality and noise issues requires robust data pre-processing, cleaning, and validation techniques (Aggarwal, 2007).

As data streams often contain sensitive information, ensuring data security and privacy is paramount. This requires the implementation of encryption during transit and at rest, authentication, and access control mechanisms to protect data while complying with data privacy regulations.

In conclusion, meeting the key requirements and addressing the challenges associated with data streaming architectures is crucial for building effective solutions and efficient processing of large-scale data streams.

2 Literature Review

The rapid growth of data generation in today's digital landscape has necessitated the development of advanced data processing techniques to derive valuable insights and drive decision-making. Data streaming architectures have emerged as a critical component in addressing the need for real-time analytics and efficient processing of large-scale data streams. This literature review aims to provide an understanding of the history, principles, technologies, and best practices associated with data streaming architectures. It will delve into the evolution of these architectures first.

The chapter is organized as follows: Section 2.1 presents an overview of the evolution of data streaming architectures, tracing their development from simple event processing systems to modern, highly-scalable, and fault-tolerant architectures. Section 2.2 presents methodology of Systematic Literature Review (SLR) of data streaming architectures which is performed in Section 2.3. Next subsections work with the results of SLR analyzing them and answering the research questions in order to achieve research objectives which were set at the beginning of the chapter.

Research Objectives

The following objectives were chosen for the literature review:

1. Define and describe the available concepts of Data Streaming Architecture.
2. Identify the common components of those architectures.
3. Define how frequent those components occur in found architectures.

Research Questions

To accomplish the objectives described above, the main question was chosen — “What are the approaches to implementing Data Streaming Architecture?”

Answering the main question will require answering the following support questions:

1. What are the concepts of Data Streaming Architecture?
2. What components occur in the documents found?
3. Is it possible to identify the common characteristics, if so, what are they?
4. Which of the defined components are the most common and which are the least common?

By conducting a thorough review of the existing literature, this chapter aims to establish a solid foundation for understanding the complex landscape of data streaming architectures.

2.1 Evolution of Data Streaming Architectures

The evolution of data streaming architectures can be traced back to the increasing need for processing vast amounts of data generated by various sources, such as social media, IoT devices, and online transactions. Over time, advancements in technology and the growing demand for real-time analytics have driven significant changes in the design and implementation of data streaming architectures.

2.1.1 Early Stages: Batch Processing and Simple Event Processing

In the early days of big data processing, batch processing was the dominant approach, with systems like Hadoop MapReduce being widely adopted. Batch processing involves collecting and processing large chunks of data at predetermined intervals, which can result in high latency and limited real-time analytics capabilities (Dunning & Friedman, 2016).

To address these limitations, simple event processing systems emerged, which focused on processing individual events or messages as they arrived (Moxey et al., 2010). These systems were often built on top of messaging systems, such as Apache ActiveMQ and RabbitMQ (Judalet, 2019), and provided basic data processing capabilities, such as filtering and routing. However, they lacked advanced features like stateful processing, fault tolerance, and scalability, which limited their applicability in complex use cases.

2.1.2 Emergence of Complex Event Processing (CEP) Systems

As the demand for real-time analytics and complex data processing grew, Complex Event Processing (CEP) systems emerged as a new generation of streaming architectures. CEP systems were designed to process multiple incoming events and identify patterns, correlations, and trends across event streams. Early CEP systems, such as Esper and StreamBase, introduced powerful features like windowing, aggregation, and pattern matching, enabling more sophisticated real-time analytics (Robins, 2010).

2.1.3 Distributed Stream Processing Systems

The advent of distributed stream processing systems marked a major milestone in the evolution of data streaming architectures. These systems, such as Apache Storm and Apache Samza, were designed to scale horizontally across multiple nodes, providing fault tolerance and high availability (Kamburugamuve & Fox, 2016). They enabled efficient processing of large-scale data streams by parallelizing tasks and distributing them across a cluster of machines.

2.1.4 Unified Stream and Batch Processing

As data streaming architectures continued to evolve, the need for a unified approach to both stream and batch processing became apparent. Systems like Apache Flink and Apache Beam emerged, offering a unified programming model and runtime that could seamlessly handle

both streaming and batch data. These systems introduced advanced features such as event time processing, stateful computations, and exactly-once processing semantics, further enhancing the capabilities of data streaming architectures (Apache, 2021).

2.1.5 Modern Data Streaming Architectures

The modern era of data streaming architectures is characterized by highly scalable, fault-tolerant, and low-latency systems that cater to a wide range of use cases. These architectures often leverage multiple components and technologies, such as Apache Kafka for data ingestion, Apache Flink for data processing, and Apache Pulsar for data storage and analytics. Modern streaming architectures also emphasize the importance of partitioning, state management, and backpressure handling to ensure optimal performance under varying workloads.

In conclusion, the evolution of data streaming architectures has been driven by the growing demand for real-time analytics and the need for efficient processing of large-scale data streams. From simple event processing to complex event processing, and from distributed stream processing to unified stream and batch processing, data streaming architectures have come a long way. Today, modern data streaming architectures offer a powerful and flexible solution for organizations looking to harness the power of real-time analytics and make data-driven decisions with confidence.

2.2 Methodology of Systematic Literature Review

A Systematic Literature Review is a rigorous and structured approach to identifying, evaluating, and synthesizing research evidence relevant to a specific research question or topic developed by (Kitchenham & Charters, 2007). In the context of this thesis, a SLR will be conducted to provide a comprehensive understanding of data streaming architectures, key requirements, and common components. This section outlines the methodology employed for conducting the systematic literature review, including the search strategy, study selection criteria, data extraction, and synthesis process.

2.2.1 Search Strategy

The search strategy for this systematic literature review involves identifying relevant research articles, technical reports, and other scholarly sources from databases and digital libraries, including IEEE Xplore, ACM Digital Library, and citational databases Web of Science and Scopus. The search is conducted using a combination of keywords and phrases related to data streaming architectures, such as “data streaming”, “data streaming architecture” and “streaming architecture”. The search strategy also includes a review of the reference lists of identified sources to uncover additional relevant studies.

2.2.2 Study Selection Criteria

To ensure the inclusion of high-quality, relevant studies in the review, the following selection criteria were applied:

- **Relevance:** studies must focus on data streaming architectures, their components, and related technologies. Studies that provide a detailed description and analysis of common data streaming architecture components will be prioritized.
- **Publication type:** only research articles, technical reports, and other scholarly sources will be considered. Editorials, opinion articles, conference papers and non-peer-reviewed publications will be excluded from the initial search query of each database.
- **Language:** studies must be published in English.
- **Publication date:** studies published within the last ten years will be prioritized to ensure the review covers recent advancements in the field. However, seminal works that have significantly contributed to the understanding and development of data streaming architectures may also be considered, regardless of their publication date.
- **Citation count:** studies with a higher citation count will be considered more relevant, as they are likely to have made a significant contribution to the field. However, this criterion will be used with caution, as newer studies may not have had enough time to accumulate citations.

2.2.3 Data Extraction

Data extraction involves collecting relevant information from the selected studies for further analysis and synthesis. Papers will be searched based on their:

- **Bibliographic details:** author(s), publication year, title, and source.
- **Study type:** research article, technical report, etc.
- **Research focus:** data streaming architectures, key requirements, common data streaming architecture components.
- **Key findings:** major contributions and insights related to data streaming architectures.

2.2.4 Meta Analysis Search

Initially, it is important to determine whether any existing meta-analyses may overlap with the subject matter of this thesis, as this could facilitate the author's process of reviewing scholarly articles.

Review begins using subject-specific databases, starting with IEEE Xplore. For this database, the author opted for the most extensive search formula, specifically ("All Metadata":data streaming architecture) AND ("All Metadata":meta-analysis), with the search timeframe commencing from 2013. This query yielded only one paper, which was relevant in terms of the topic, but it was not meta-analysis.

The subsequent subject-specific database to explore was the ACM Digital Library, a textual repository of articles from the Associated Computing Machinery journal, which focuses on computer science. In this instance, the search query was similar to the previous one, but with adjustments for the distinct syntax used by this database: [Full Text: "data streaming architecture"] AND [Full Text: "meta-analysis"] AND [E-Publication Date: (01/01/2013 TO *)]. The search yielded no relevant papers.

The subsequent search resource to consider is the Web of Science database, specifically its Web of Science Core Collection. The search utilized the following formulas: (ALL=("data" AND "streaming architecture" AND "meta-analysis")) and (ALL=("data streaming architecture" AND "meta-analysis")). In both cases, the database was not able to locate any relevant publications.

The final database explored for meta-analyses was the Scopus database, which was searched across all categories without any subject limitations except those outlined in the query. The formula "ALL ("data streaming architecture" AND "meta-analysis")" was employed for the search, with an added constraint of a publication date post-2013. This search yielded 2 publications, one of which was generally related to this paper but not a meta-analysis itself.

Results of the search are presented in the Table 1. The author suggests that the limited number of discovered publications can be attributed to the fact that the subject of data streaming architectures is relatively novel and continuously evolving. As a result, no prominent researchers have yet consolidated the work on this topic in a meta-analysis.

Table 1: Number of found meta-analyses per database. Source: author

Database	Publications found	Relevant publications
IEEE Xplore	1	0
ACM Digital Library	0	0
Web Of Science	0	0
Scopus	2	0

2.2.5 Scientific Papers Search

Given that the search for meta-analyses did not yield relevant results, it becomes essential to seek out relevant individual scientific papers and articles published. This search will be conducted adhering to the criteria outlined in section 2.2.2, while also accounting for the unique characteristics of various databases and their respective query formats.

Analysis

The analysis will utilize the keywords specified in section 2.2.1, with the search process incorporating various combinations of these terms.

Initially, the IEEE Xplore database was searched, yielding 39 papers (query formula was: (“All Metadata”:”data streaming”) AND (“All Metadata”:”architecture”)) with additional filters to exclude conference papers and publications that are more than 10 years old). The ACM Digital Library database revealed 123 papers (query formula: “[All: ”data streaming”] AND [All: ”architecture”] AND [E-Publication Date: (01/01/2013 TO 04/30/2023)]”), while the Web of Science database uncovered 91 papers (query formula: “(ALL=(”data streaming” AND ”architecture”))” with filters similar to IEEE Xplore formula). Lastly, the Scopus database, using the same keywords, identified 74 papers (query formula: “ABS (”data streaming” AND ”architecture”)” additional filters were applied to filter publication year and exclude irrelevant conference papers). The search results showed a higher number of works discovered in the ACM Digital Library database. The author posits that this figure in this specific database is a result of its search algorithm.

During the data synthesis phase, a thorough examination of the search outcomes was conducted. For all databases, this cleanup process entailed manually reviewing each paper's abstract and assessing its content for relevance to the current study. Results are presented in Table 2.

Table 2: Number of chosen publications per database. Source: author

Database	Publications found	Relevant publications without duplicates	Publications chosen to review the results	Publications selected after review of the reference lists
IEEE Xplore	39	34	0	2
ACM Digital Library	123	123	1	1
Web Of Science	91	85	1	1
Scopus	74	67	0	1

The initial search results enabled the author to discover two relevant articles suitable for inclusion in this paper. However, following the methodology outlined in section 2.2.1, the analysis also includes articles not chosen for review in this study. During the investigation of the reference lists in these not-relevant articles, an additional 5 relevant sources were identified that fulfill the criteria for this paper.

2.3 Architectures Identification

In the subsequent section, the identified articles and the architectures described by their authors will be analyzed in the context of the questions raised by the researcher at the start of the study. The architectures are organized chronologically, starting with the earliest publications, and progressing to the most recent ones. Upon examining the architectures, a consolidation of shared architectures components is to be presented.

2.3.1 Kafka Usage at LinkedIn Architecture

The publication “Kafka: a Distributed Messaging System for Log Processing” presents the Kafka framework, which was new at the time of publication. The primary focus of the publication is on Kafka's applications in log processing, aggregation, and utilization. The paper also compares Kafka to the previously existing ActiveMQ and RabbitMQ.

The authors also define the components of the Kafka framework. By defining the components. These components include:

- **Brokers:** central components of Kafka that receive messages from producers and store them in partitions. They are responsible for managing the data replication and partitioning of messages.
- **Topics:** refers to a category or feed name to which messages are published by producers. Each partition is assigned to a specific broker and can have multiple replicas for fault tolerance.
- **Partition:** is a logical unit of data that stores a sequence of messages in Kafka. Messages within a partition are ordered and assigned a unique offset value, which can be used to retrieve them.
- **Producers:** are responsible for publishing messages to Kafka topics. They can send messages synchronously or asynchronously and can choose which partition to publish the message to.
- **Consumers:** are responsible for reading messages from Kafka topics. They can read messages from one or more partitions and can maintain the order of messages within a partition.

Of particular interest in the paper is how Kafka is utilized at LinkedIn, the organization where the tool was developed. The authors describe a simplified architecture (shown at Figure 1) that is implemented at their company. The architecture involves frontend services that produce different types of logs, which are then sent to Kafka brokers. Interestingly, this process is performed in batches. A hardware load balancer is then used to distribute the requests for publishing messages evenly across the group of Kafka brokers. The data is subsequently extracted from the brokers for further processing, both online and offline. For online analysis, the same data center is used, whereas for offline analysis, another cluster is established in a different data center, closer to the company's warehouse architecture. The data is copied to this cluster for additional analysis (Kreps et al., 2011).

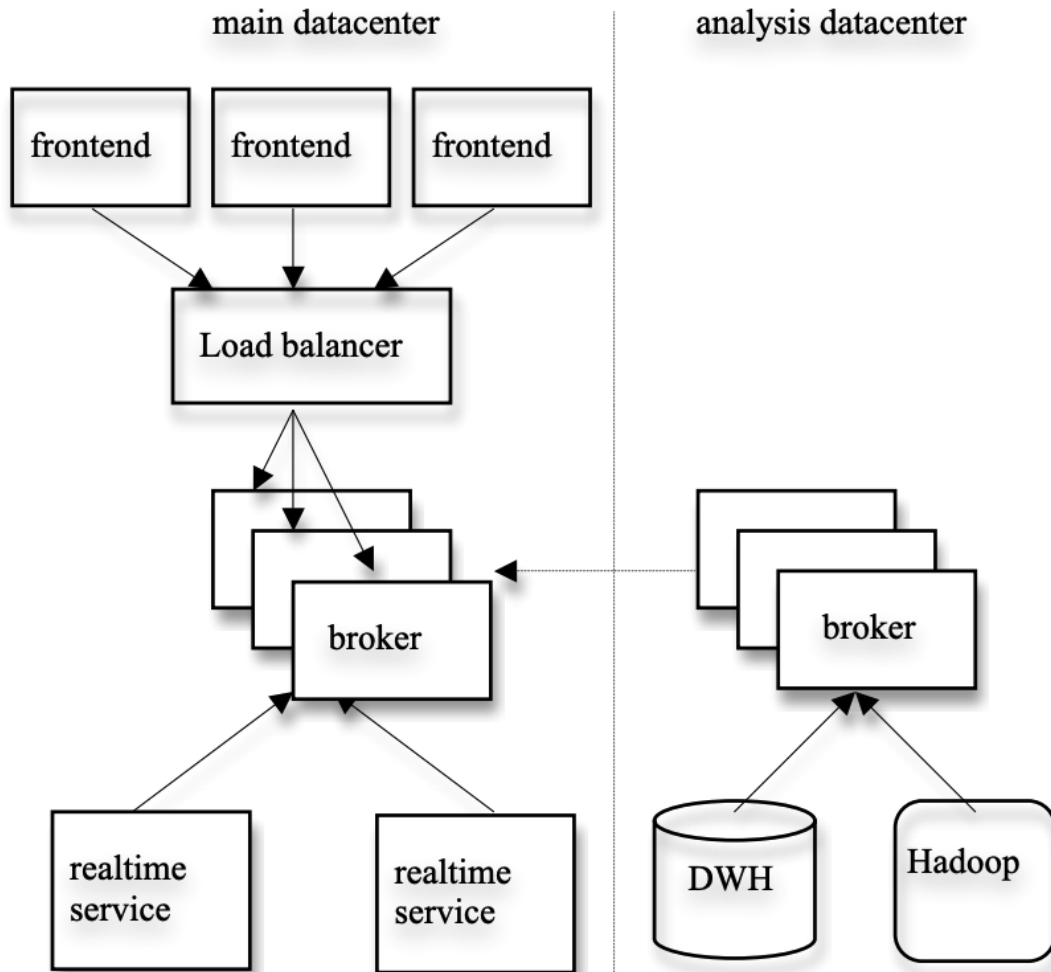


Figure 1: Kafka Deployment at LinkedIn. Source: (Kreps et al., 2011)

To ensure that there is no loss of data throughout the entire pipeline, they have an auditing system in place as part of the tracking process. To accomplish this, each message includes a timestamp and the name of the server where it was generated. Architecture also instruments each producer with the ability to periodically generate a monitoring event. These events record the number of messages published by each producer for each topic within a fixed time window (Kreps et al., 2011). The authors of the publication do not explicitly mention any aggregations in the data flow architecture. Instead, all the aggregations in this architecture occur at the end of the stream, which is at the precise moment when the data is written to the offline warehouse cluster, utilizing a MapReduce job.

2.3.2 Concept of Global Design for Streaming Architecture

The next architecture chosen for the review is the one from the book “Streaming Architecture” by (Dunning & Friedman, 2016). In general, the book explores the concepts and principles of modern streaming architecture, focusing on Apache Kafka and MapR Streams as key technologies. The book delves into the design and implementation of streaming data systems, discussing the critical role of these systems in modern data-driven applications.

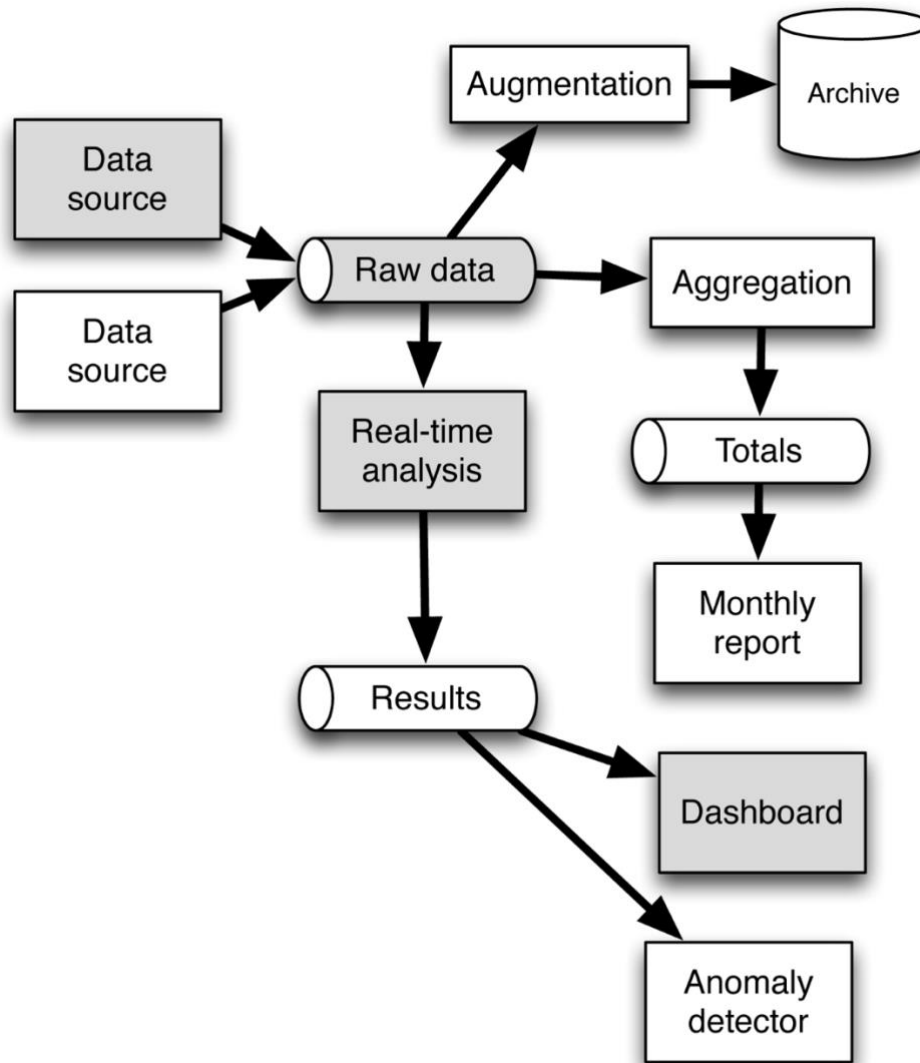


Figure 2: Concept of Global Design for Streaming Architecture. Source: (Dunning & Friedman, 2016)

In this analysis, we will focus primarily on the chapter titled “Key Aspects of a Universal Stream-based Architecture.” In this chapter, the authors build upon the theoretical foundation laid out in earlier sections of the book and attempt to propose a universal streaming architecture concept. This concept, as defined by authors, should be versatile enough to be adapted to different specific cases, such as data streams consumed directly by dashboards. The authors establish several requirements for this architecture, including durability, which they define as “... data can be used immediately upon ingestion, but it should not disappear if the downstream process is not ready for it when the data arrives. Messages should be durable.” Furthermore, they assert that “... these architectures need to handle very large volumes of data, so the tools used to implement them need to be highly scalable throughout the system.”

The proposed architecture by the authors (as depicted in Figure 2) consists of numerous components, most of which can be readily understood in terms of their function within the structure, so they won’t be covered in this thesis. This architecture bears similarities to the one outlined by (Kreps et al., 2011), but there are also some distinctions. The most notable aspect of this architecture is the authors' unique perspective, as they emphasize that the

primary objective of the architecture “... is to use data streams throughout your overall architecture - data streaming becomes the default way to handle data rather than a specialty.” This viewpoint leads to the inclusion of a “Results” component in the architecture, which enables the utilization of analytical outcomes across multiple components (consumers) instead of writing the results directly to a single consumer.

Besides these general requirements the authors also point out the general features the implementation of this architecture should comply with, in particular the fault tolerance, i.e. the architecture should always know if the data it works with is good, it should also correctly distribute the load by load balancer, the data should have multiple copies so that the outage of one of the streams does not end up in a total loss of the message - mirroring, also data should be serialized using some algorithm.

2.3.3 Streaming Data Architectural Blueprint

We will now examine the book “Streaming Data: Understanding the Real-Time Pipeline” by (Psaltis, 2017). This comprehensive guide delves into the principles, technologies, and best practices related to building and managing real-time data processing pipelines. It provides valuable insights into the realm of streaming data.

Chapter 1.3, titled “The Architectural Blueprint,” is of particular relevance for this thesis. In this chapter, the author presents a general architecture (as illustrated at Figure 3) that “... enable us to talk about all streaming systems in a generic way...” This architecture is comprised of distinct tiers that follow and connect to one another. Generally speaking, it bears resemblance to the architectures proposed by (Kreps et al., 2011) and (Dunning & Friedman, 2016), which have already been analyzed. One notable advantage of this architecture is its broader scope and simplicity compared to others. In this case, we do not delve into specific architectural examples, but rather focus on the bigger picture, setting it apart from proposals like (Dunning & Friedman, 2016) architecture.

The authors elucidate the nature of these tiers using Twitter as an example. They describe tiers as following:

- Collection tier — When a user submits a tweet, it is gathered by Twitter's services.
- Message queuing tier — It is likely that Twitter operates data centers worldwide, and the collection of a tweet might not occur in the same location as its analysis.
- Analysis tier — Although the 140-character tweets undergo a considerable amount of processing, Twitter needs to identify the followers of a tweet.
- Long-term storage tier — Although this optional tier will not be discussed extensively in this book, it can be assumed that historical tweets are stored in a persistent data storage system.
- In-memory data store tier — Tweets that are only seconds old are probably stored in an in-memory data store.
- Data access — All Twitter clients must connect to the Twitter service to access it.

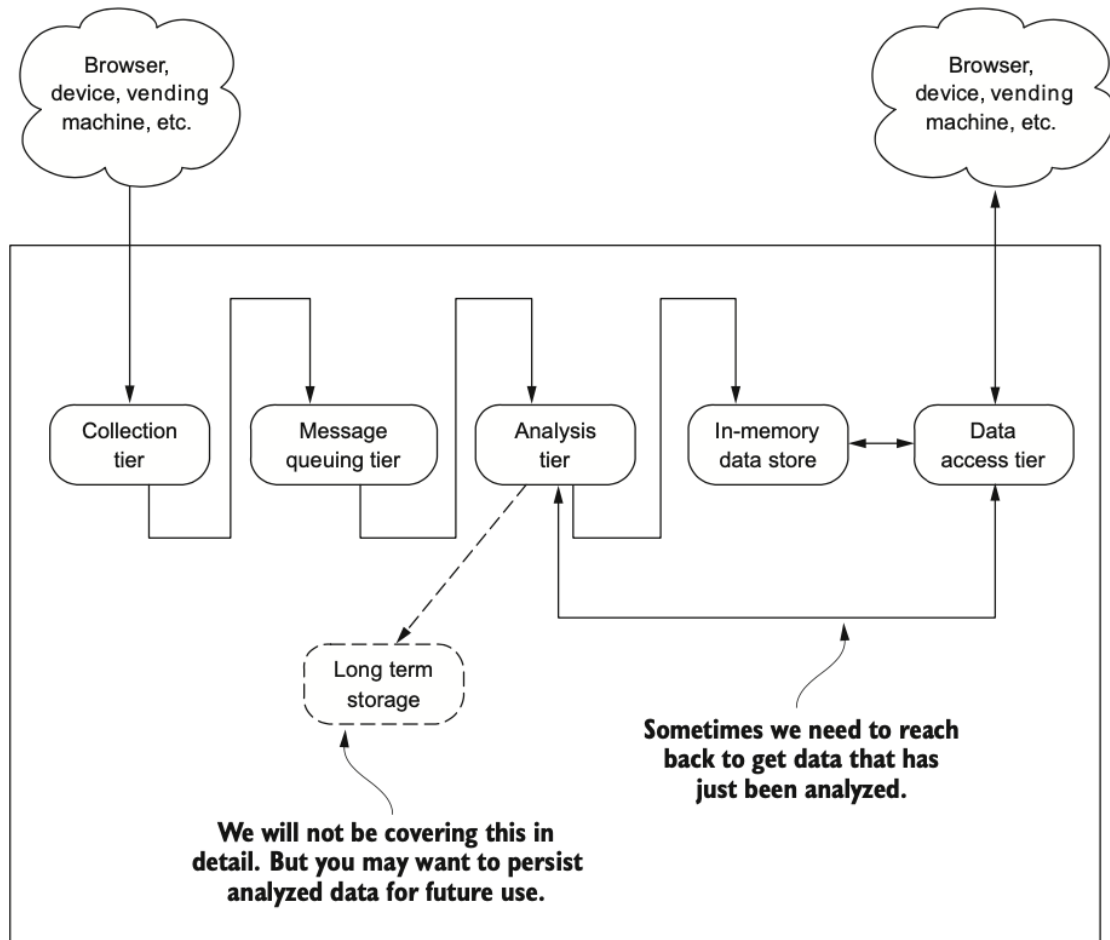


Figure 3: The Streaming Data Architectural Blueprint. Source: (Psaltis, 2017)

The author proceeds to dissect each aspect of this “blueprint” in adequate detail and outlines the components that can be incorporated when deploying it in a real system. For instance, elements such as load balancers, mirroring, stream processor and alerting are identified, just as in previous architectures (Psaltis, 2017).

2.3.4 ETL Streaming Architecture

Now let’s examine the architecture suggested by (Meehan et al., 2017) in theirs “Data Ingestion for the Connected World”. The paper primarily discusses data streaming in ETL pipelines and highlights that modern ETL pipeline architectures are more fitting for present demands. The authors express their objective as: “... to build a new architecture for ETL which takes advantage of the push-based nature of a stream processing system,” and “... describe our implementation of streaming ETL using a scalable messaging system (Apache Kafka), a transactional stream processing system (S-Store), and a distributed polystore (Intel’s BigDAWG)”. Our focus will be on the architecture proposed by the authors and the implementation aspects relevant to this paper.

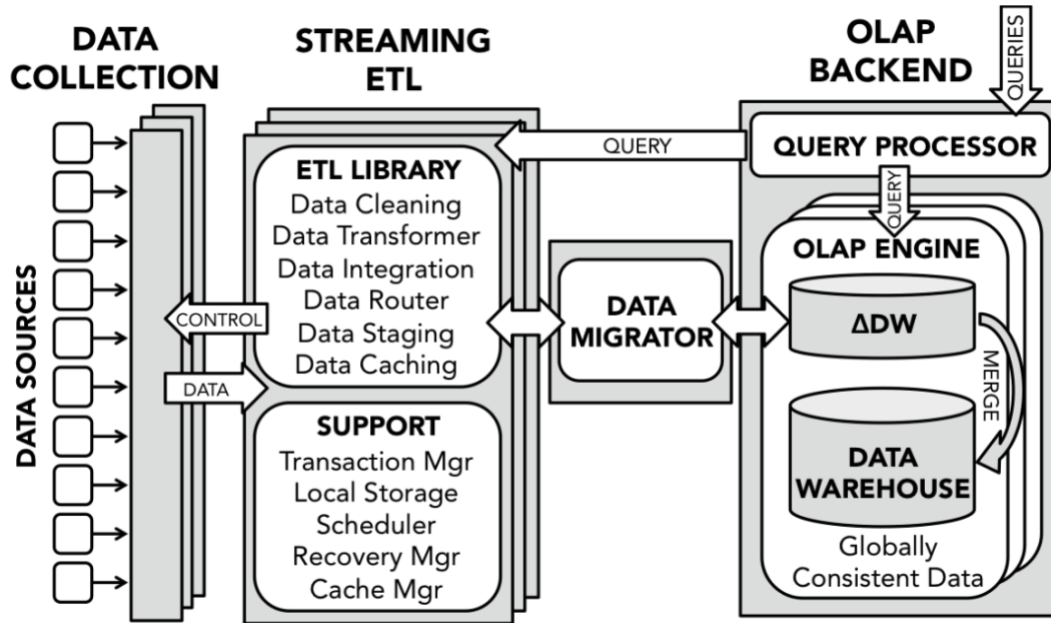


Figure 4: Streaming ETL Architecture. Source: (Meehan et al., 2017)

In the section titled “THE NEW ARCHITECTURE”, that Figure 4 displays, the proposed architecture is shown, with the authors' primary intent being “... to create a generic design that can suit a variety of data ingestion situations.” The architecture consists of four main components:

- **Data Collection** – a component that functions as the data source for the entire system, being responsible for ingesting data from various origins. The authors mention that “... data collector primarily serves as a messaging queue”, they also point, that it should be scalable, fault-tolerant, and distributed.
- **Streaming ETL** – characterized as “... streaming ETL engine features a full library of traditional ETL tools, including data cleaning and transformation operators”. Although this component is highly use-case specific, its primary role is to clean and process data for subsequent analysis. As seen in Figure 4, it encompasses numerous sub-components.
- **OLAP Backend** – as described by the authors, “OLAP backend consists of a query processor and one or several OLAP engines.” Since basic data operations occur in the previous component, this stage only requires setting up the appropriate OLAP tool, potentially including an integrated data warehouse for offline data mining or suitable query processor, that support querying stream data directly from “Streaming ETL” component without it being loaded into its integrated DW.
- **Data Migrator** – its main purpose, as pointed by the authors of the article, is to establish a dependable connection between the ETL and OLAP components, because a data is being frequently moved between “OLAP Backend” and “ETL Streaming” it also should implement a mechanism, that insure, that there is no data lost in transit.

Regarding the suggested architecture, it incorporates components established in earlier works reviewed. The new features are more implementation ones, for example, the authors implement in the architecture push-based approach, which shows itself well in the tests conducted by the authors in the architecture in the article. Otherwise, similar common features in the architecture can be distinguish.

2.3.5 Architecture For Decision Support

The subsequent paper by (Ge et al., 2019), titled “A Scalable Framework for Multilevel Streaming Data Analytics using Deep Learning.” presents an architectural perspective, where the data from the “Data Ingestion” phase will be utilized for real-time model training and integrating offline analytics for deeper data insights. Generally, the authors refer to the proposed method as a “multilevel streaming text data analytics framework.” with the term “multilevel” indicating that the architecture operates with both online data, retraining the model based on incoming data, and data stored in offline repositories for further analytics, such as data warehouses.

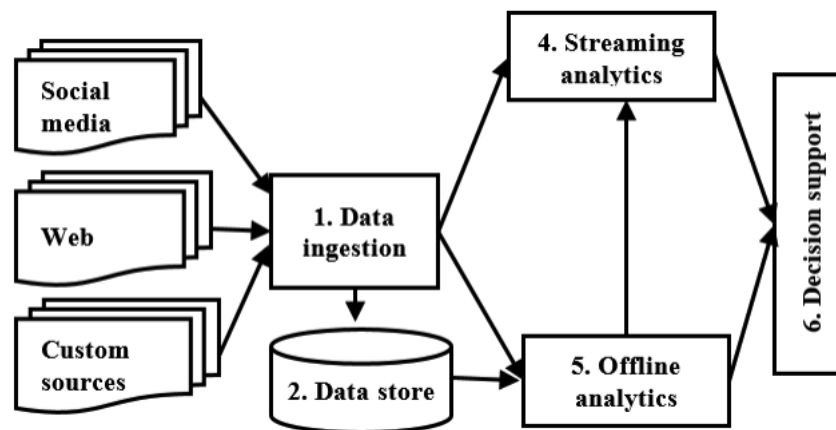


Figure 5: Architecture of the multilevel streaming analytics framework. Source: (Ge et al., 2019)

The authors present an architecture in Figure 5, emphasizing five key components, some of which have been previously encountered in other works and discussed within this study. We will concentrate on those that are not yet thoroughly understood.

- Data ingestion refers to the part of the architecture responsible for receiving input media that generates streaming data in various formats. The article's authors also note that data streams from different sources can be ingested into a data store or directly into a streaming analytics system which is similar to what we already seen in previous papers.
- The data store component serves as the storage layer for data, allowing it to be used later in offline analytics.
- Offline analytics involve analyzing previously stored data, primarily used for large datasets that cannot be processed within a streaming analytics system without first being saved.

- Streaming analytics, similar to offline analytics, focuses on real-time scoring of incoming data that can be immediately utilized for supplementary model training without intricate calculations.
- The decision support component encompasses real-time alerts or visualizations as part of its functionality.

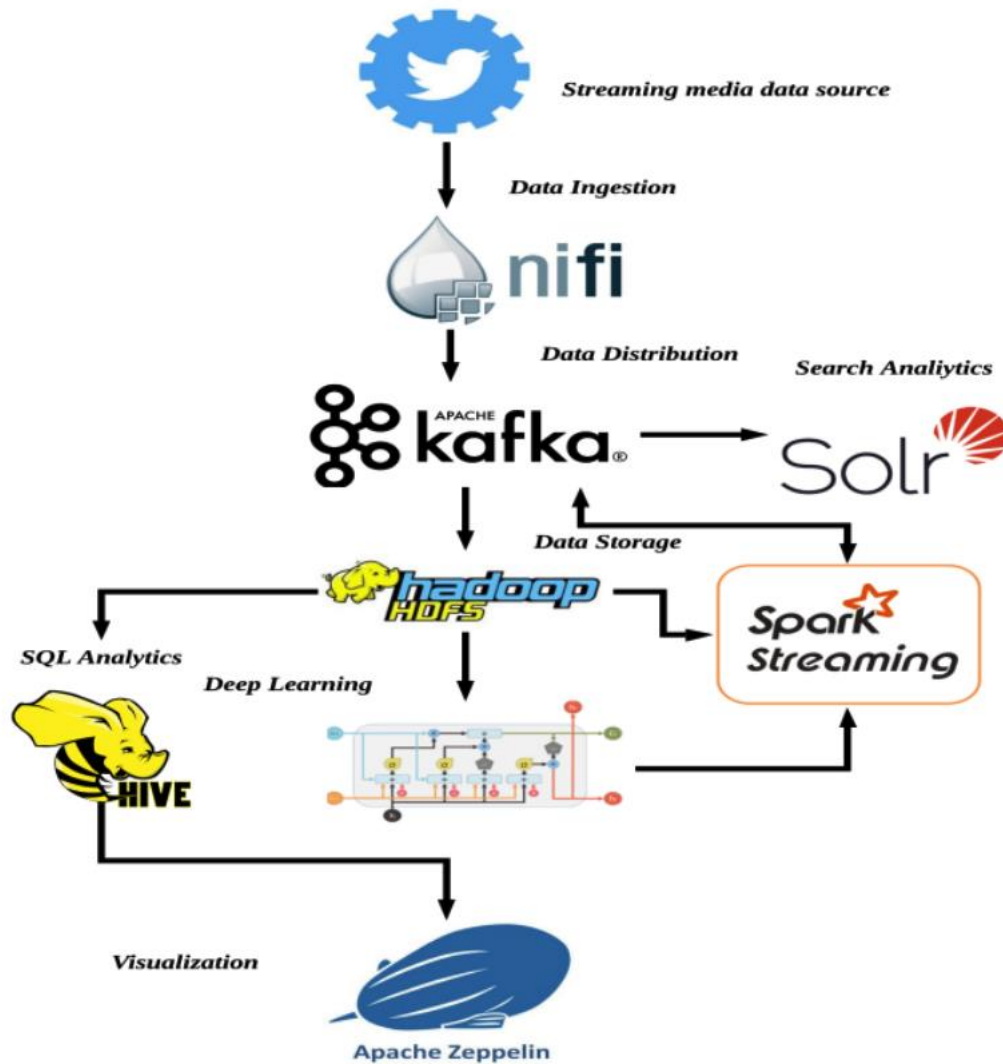


Figure 6: Implementation of the framework. Source: (Ge et al., 2019)

Overall, the architecture put forth by the authors does not deviate significantly from those previously reviewed in this thesis. Among the more interesting aspects of the authors' proposal is their approach to organizing offline and online data, not only for dashboard data analysis, as exemplified by (Dunning & Friedman, 2016) but also for model training. This means the same data is utilized for both training new models and refining existing ones, as demonstrated in the implementation (Figure 6).

2.3.6 General Architecture of a DSPS

The upcoming article to be examined in this thesis is “A Survey of Distributed Data Stream Processing Frameworks” by (Isah et al., 2019). The paper's focus is the advantages and

disadvantages of existing data streaming frameworks. However, the authors also mention, that, “[The study] also reports our ongoing study on a multilevel streaming analytics architecture that can serve as a guide for organizations and individuals planning to implement a real-time data stream processing and analytics framework.” In this study, the section outlining the proposed architecture is of utmost importance. Before going to the paper, the frequently used acronym DSPS should be emphasized, DSPS is Data Stream Processing System.

In the “COMPONENTS OF A DSPS” section, the authors detail the architecture components, and it is worth noting that part of the paper is based on previously reviewed work from (Meehan et al., 2017). The authors initially observe that “... despite the diverse data domains and business logic, a generic data processing pipeline exists...” and then present their architectural perspective.

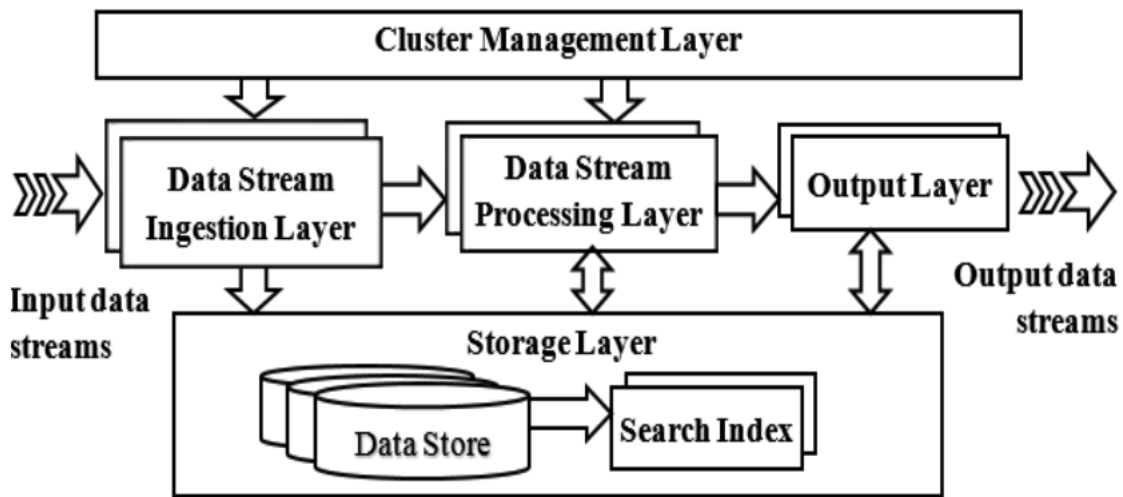


Figure 7: Architecture of a Data Stream Processing System (DSPS). Source: (Isah et al., 2019)

Figure 7 displays the proposed architecture and its components. The authors describe each layer of the architecture as follows:

- The ingestion layer guarantees scalable, resilient, and fault-tolerant data distribution across the DSPS architecture from various input data streams while decoupling the input data sources from other parts of the DSPS.
- The data stream processing layer is where streaming data processing applications or jobs are executed. It can host loosely coupled disjoint applications, a DSPE, or both. DSPEs typically provide a set of streaming data processing operators that can be configured and connected to create a data processing pipeline for analyzing incoming data streams.
- DSPSs often store analyzed data, identified patterns, and extracted knowledge from different data processing stages for further processing. Stored data must be organized and indexed, along with external knowledge or metadata, for executing subsequent analytics jobs.

- The resource management layer orchestrates actions among computing and storage nodes, managing resource allocation and scheduling in distributed systems to enable parallel processing of high volume and velocity data streams.
- The outcomes from data stream processing pipelines can be directed to an application, another workflow, a visualization tool, or an alert or monitoring dashboard. Concurrently, data and extracted knowledge can be transferred to temporary in-memory or permanent storage for further analysis and querying.

Generally speaking, the proposed architecture shares similar features and consistent logic with others presented in this thesis. Interesting aspects suggested by the authors include resource management in a separate layer, whereas in other works, this part of the architecture is considered integrated thus not talked about. Here, the authors designate it as a distinct component. We can also observe the output layer, which essentially is the generalization of the more specific results of the pipelines proposed by (Ge et al., 2019) and (Dunning & Friedman, 2016).

2.3.7 Upsolver Reference Architecture

The most recent architecture is suggested by (Upsolver, 2022) in their online publication titled “An Architect's Guide to Stream Processing at Scale”. Similar to (Isah et al., 2019) the primary focus of this publication is to explore available implementation options and outline their advantages and disadvantages. However, in the final section of the publication called “Future-proofing Your Architecture with Upsolver”, a synthesis of the mentioned components is presented as a generalized system architecture. This architecture can be viewed in Figure 8.

In comparison to other proposed architectures, there is not much to add about this specific architecture, as it also consists of the main layers we have previously observed in other architectures and articles.

While the authors use distinct names for their components, there are no fundamental differences in their approach to architecture. Instead, these names represent the authors' conceptual views on how the architecture should appear. For instance, “Data Lake Storage” which has been referred to as “Data Storage” in other articles, is explained by the authors as a response to the limitations of traditional data warehouses: “... large enterprises have used data warehouses to mine the value hidden in their data. But warehouses can't keep up with all data processing needs (streaming, machine learning, exponential growth in volumes).” In this case, the choice of name and specific solution is dictated by preference rather than being based on specific comparisons. As such, it can be summarized as a component within the already familiar “Data Storage”.

Nonetheless, it is worth mentioning in this paper due to its relevance to the topic of the thesis.

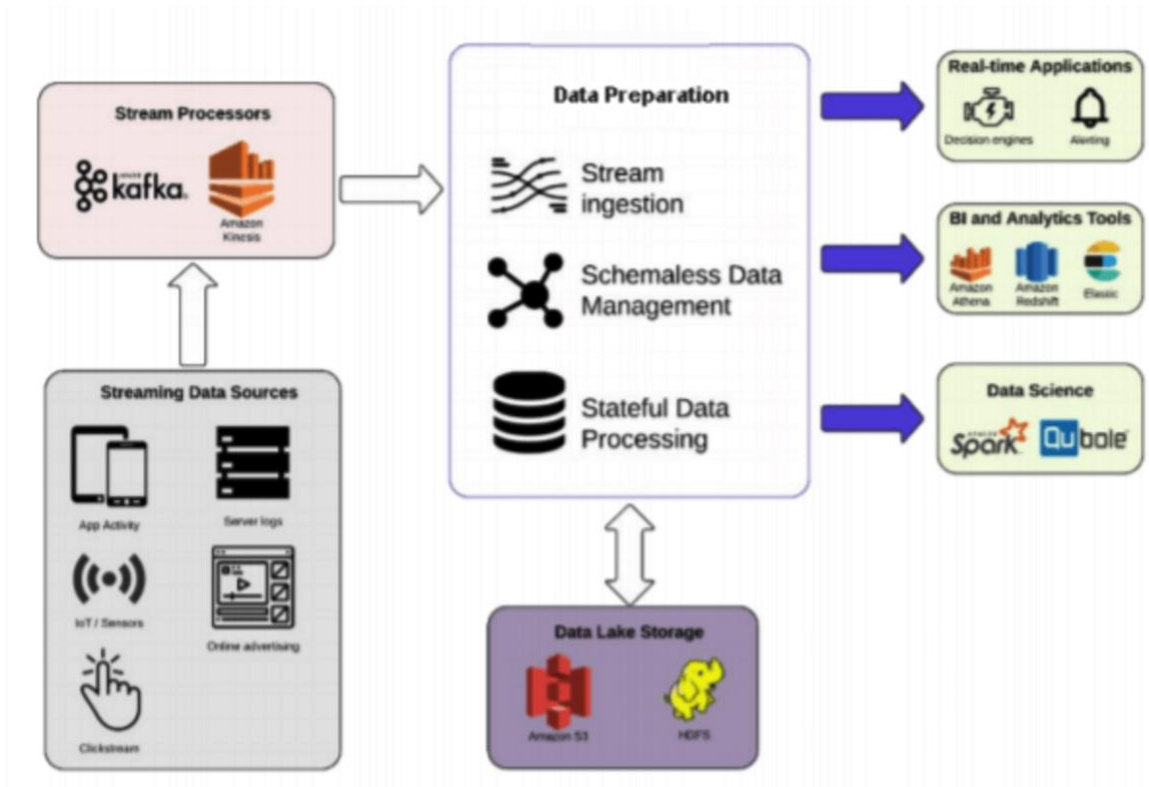


Figure 8: Reference Stream Processing Architecture. Source: (Upsolver, 2022)

2.4 Synthesis of Described Components

In the previous subsection, seven distinct architectures suggested by theirs' authors were examined, the author had assessed them broadly and identified shared characteristics in each case. This chapter was necessitated due to the authors of reviewed works employing varying terms for identical concepts, such as "Data Storage" as explained in subsection 2.7.

In the current section, the discovered components are consolidated, and the attempt to apply induction and generalization to establish a unified terminology for subsequent analysis.

Table 3: Synthesis of architectures' components 1-4. Source: author

Suggested architecture layers	Kafka usage at LinkedIn Architecture (2.3.1)	Concept of Global Design for Streaming Architecture (2.3.2)	Streaming Data Architectural Blueprint (2.3.3)	ETL Streaming Architecture (2.3.4)
Data Source Layer	Frontend	Data Source	Browser, Device, Vending Machine, etc.	Data Sources

Collection Layer	Kafka Brokers	Messaging System	Collection Tier	Data Collection
Stream Processing Layer	–	Real-Time Analysis	Analysis Tier	Streaming ETL
In-Memory Storage Layer	Main Datacenter	Results (Real-Time Analysis)	In-Memory Data Store	Streaming ETL (Data Caching)
Long-Term Storage Layer	Analysis Datacenter, DWH, Hadoop	Archive (Augmentation)	Long-Term Storage	Data Warehouse
Output Layer	Realtime Service	Dashboard	Data Access Tier	OLAP Backend
Analytics Layer	Reporting and Analytical Jobs	Monthly Report	–	OLAP Backend (Query Processor)
Data Quality Layer	Auditing System	Anomaly Detector	–	Data Migrator

Table 4: Synthesis of architectures' components 5-7. Source: author

Suggested architecture layers	Architecture For Decision Support (2.3.5)	General Architecture of a DSPS (2.3.6)	Upsolver Reference Architecture (2.3.7)
Data Source Layer	Twitter Streaming API	Input Data Streams	Streaming Data Sources
Collection Layer	Data Ingestion	Data Streaming Ingestion Layer	Stream Ingestion
Stream Processing Layer	Data Distribution	Data Streaming Processing Layer	Stream Processors
In-Memory Storage Layer	–	Data Streaming Processing Layer (ETL Engine for Real-Time Query Processing)	–
Long-Term Storage Layer	Data Storage	Storage Layer	Data Lake Storage
Output Layer	Real-time analytics	Output Layer	Real-Time Applications

Analytics Layer	SQL Analytics	Storage Layer (OLAP Backend)	BI and Analytics Tools
Data Quality Layer	–	–	–

2.5 Occurrences of Components in Architectures

This section examines the frequency at which the recognized elements appear in various architectures analyzed. Initially, a table is provided, illustrating the components alongside their respective architectures, with the intersections indicating the presence of a component in a specific architecture (Table 5). Subsequently, a graph is displayed, highlighting the most and least prevalent components (Figure 9).

For ease of presentation in the following table and graph, the components will be presented under the following abbreviations:

- DSO – Data Source Layer
- COL – Collection Layer
- SPR – Stream Processing Layer
- IMS – In-Memory Storage Layer
- LTS – Long-Term Storage Layer
- OUT – Output Layer
- ANS – Analytics Layer
- DAQ – Data Quality Layer

Table 5: Components occurrences in architectures. Source: author

Architecture	DSO	COL	SPR	IMS	LTS	OUT	ANS	DAQ
Kafka usage at LinkedIn Architecture (2.3.1)	×	×		×	×	×	×	×
Concept of Global Design for Streaming Architecture (2.3.2)	×	×	×	×	×	×	×	×
Streaming Data Architectural Blueprint (2.3.3)	×	×	×	×	×	×		
ETL Streaming Architecture (2.3.4)	×	×	×	×	×	×	×	×
Architecture For Decision Support (2.3.5)	×	×	×		×	×	×	
General Architecture of a DSPS (2.3.6)	×	×	×	×	×	×	×	

Upsolver Reference Architecture (2.3.7)	×	×	×	×	×	×
--	---	---	---	---	---	---

With Table 6, it becomes evident that the “Concept of Global Design for Streaming Architecture” and “ETL Streaming Architecture” align with the suggested common architecture most closely, while other architectures frequently miss one or more components. The least compatible with the proposed architecture are “Streaming Data Architectural Blueprint” and “Upsolver Reference Architecture”.

Now, let's examine the frequency of components' occurrence in the considered architectures shown in the Figure 9. It is apparent that elements like DSO, COL, LTS, and OUT are present in all architectures. Consequently, we can infer that these elements are indispensable, no matter how the architecture is approached from an implementation perspective. These components constitute the primary stages of data lifecycle, initially being gathered from the stream, and then stored or forwarded for further processing. All other components can be deemed optional, appearing when required on case-by-case basis.

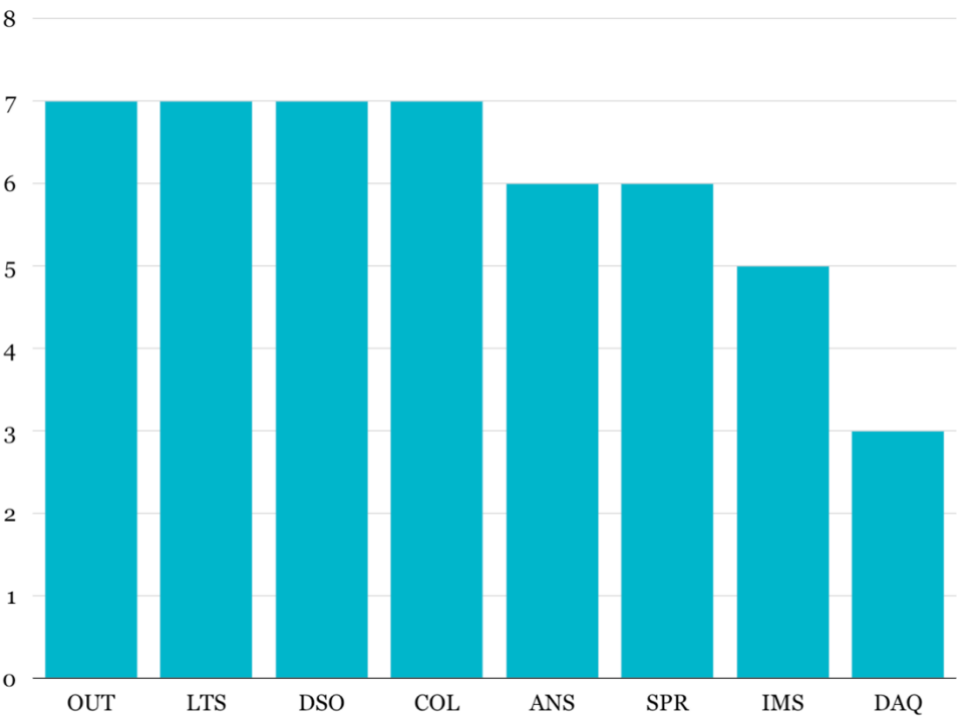


Figure 9: Frequency of components' occurrence. Source: author

2.6 Discussion

This section of the thesis addresses the analysis of the findings from the comprehensive literature review conducted in this chapter. Additionally, it explores the research questions

presented initially and the outcomes derived from those questions. The limits of the conducted review are also noted.

2.6.1 Research Questions of the Systematic Literature Review

In Section 2.1, research questions were established to achieve the thesis’s objectives. The responses to these questions satisfied the research objectives outlined in Table 6.

Table 6: Research objectives of the Systematic Literature Review and their status. Source: author

Research objective	Status
Define and describe the available concepts of Data Streaming Architecture.	Met in the chapter 2.3
Identify the common components of those architectures.	Met in the chapter 2.4
Define how frequent those components occur in found architectures.	Met in the chapter 2.5

The initial two objectives focused on the components of big data architecture and the concepts discovered in the examined research papers. The first research objective, connected to these questions, was addressed in Section 2.3 by describing the found architectures and their components.

The subsequent query pertained to the similarities among the identified components. This question and its associated research objective were addressed in Section 2.4, particularly in Tables 3 and 4, where the author summarizes the discovered components and their shared features.

The final question sought to determine the frequency of the identified components in the reviewed architectures, identifying the most common and least common occurrences. This was addressed in Section 2.5, specifically through Table 5 and Figure 9.

All of the above objectives have been identified to fulfill the aim of the thesis, which will be completed in Chapter 3. A general architecture based on the found and identified components will be proposed.

2.6.2 Limits of Research Conducted

Before presenting the overall architecture and findings of this study, it is essential to acknowledge the main limitations. Some limitations include the criteria for selecting works, as outlined in Chapter 2.2.

For instance, although the search was conducted in four major databases, with one being highly specialized in computer science, it is possible that numerous scientific papers were not included simply because they were not present in these databases. Nevertheless, to

ensure a more comprehensive review of architectures, the author examined the sources of relevant articles, extending the search beyond the specified databases and discovering five additional relevant papers.

The publication type, language, and publication date also posed limitations. The publication type was somewhat restricted by examining sources in relevant articles, while the author imposed other limitations for practical reasons. However, the existence of relevant works beyond the defined criteria cannot be ruled out.

Furthermore, it is important to consider the influence of the specific queries used to search for relevant works in the databases. The phrasing itself can significantly impact the results and likely affected the outcome. Therefore, author cannot say that all relevant works in the above databases were found.

Lastly, one of the most critical limitations is the subjective evaluation of the relevance of the discovered works. As shown in Table 2, out of the 327 works found, the author deemed only seven to be relevant and included them in this study. It is possible that another researcher might have chosen different works based on the criteria chosen in this thesis.

3 General Data Streaming Architecture Proposal

By employing the scientific methodologies outlined at the beginning of the thesis, the author presents their proposal of the general Data Streaming Architecture. This chapter also elaborates on the proposed architecture, its components, and their descriptions. The scientific methods used to examine the works and develop the subsequent general architecture encompass analysis, induction, generalization, Systematic Literature Review, and synthesis. The systematic literature review in Chapter 2.3 helped identify the papers that were later analyzed. From these papers, induction and generalization in Chapter 2.4 were employed to pinpoint and generalize the components, while synthesis was utilized to integrate these components into the architecture proposed in this chapter.

While the suggested general architecture does not claim to be universally applicable, the author believes that, based on the analyzed literature, it can be considered sufficiently comprehensive and incorporates all necessary general components identified in this paper.

Subsequently, a graphical representation of the proposed architecture is shown in Figure 10. Concerning the symbols in the figure, rectangles signify a process in the data stream, while cylinders represent data storage or sources in any form. A diamond shape symbolizes a group of procedures that, as they are connected to all underlying layers, are not linked to other layers with lines. Storage layers are indicated at the bottom, and a dashed line traces the path to it since its implementation is not essential, and a pipeline could potentially function without it. Moreover, both storage types are enclosed by a rectangle that encapsulates their function, and an arrow extends from it to the “Analytics Layer”, signifying that analytics cannot occur without persistent data. Similarly, a dotted arrow connects the “Output Layer” to the “Analytics Layer,” indicating that the “Analytics Layer” is optional, and post-stream processing data can proceed through the pipeline. Now let's move on to the description of the components.

3.1 Data Source Layer

The “Data Source Layer” represents the starting point where raw data is generated and emitted from various sources before entering the “Collection Layer”. As the first layer in the pipeline, it is crucial for capturing a wide range of data types and formats, ensuring a comprehensive and diverse data stream to be processed and analyzed further down the pipeline.

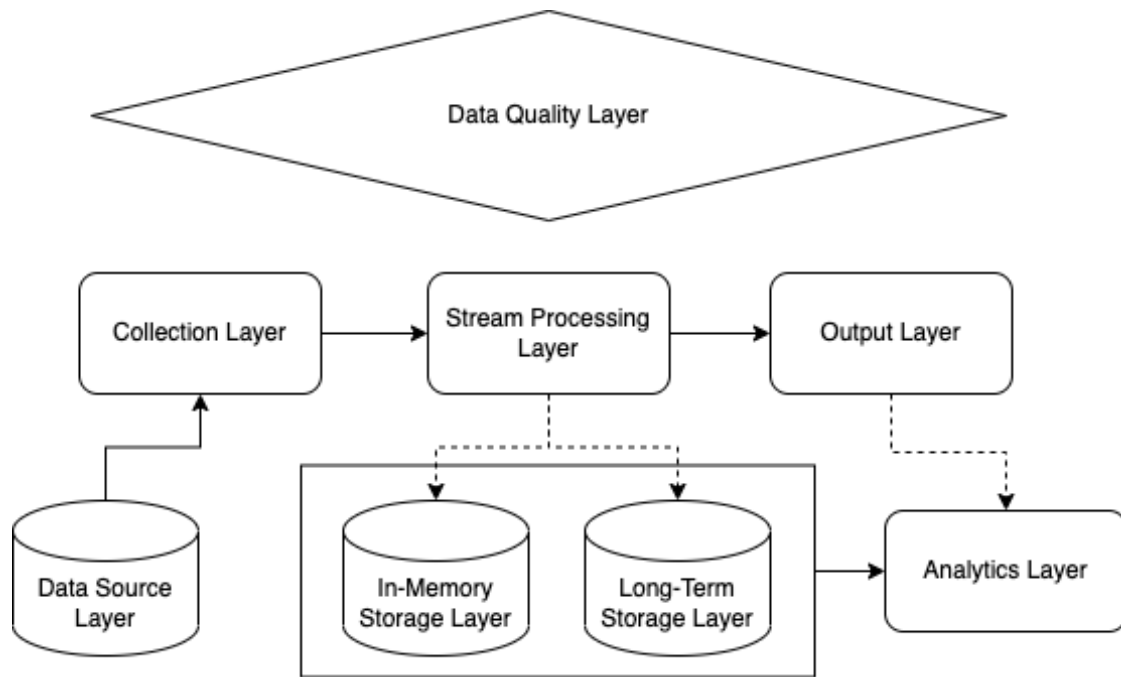


Figure 10: General Data Streaming Architecture. Source: author

3.2 Collection Layer

The “Collection Layer” in this pipeline serves as the initial stage responsible for gathering, ingesting, and preprocessing raw data from various sources. This layer plays a crucial role in ensuring seamless and efficient data flow throughout the entire pipeline. It collects data from sources such as sensors, logs, social media feeds, or other applications, and then normalizes and filters it to ensure consistency and relevance before passing it on to subsequent layers.

3.3 Stream Processing Layer

The “Stream Processing Layer” is the crucial stage where real-time data processing and transformation occur. Positioned after the “Collection Layer”, it receives the preprocessed and normalized data to perform various operations, such as filtering, aggregation, and enrichment, before passing it on to subsequent layers. This layer should be designed to handle high-velocity data streams and deliver low-latency processing results.

3.4 Output Layer

The “Output Layer” is the stage where processed and transformed data from the “Stream Processing Layer” is delivered to various target systems or applications for further consumption. This layer plays a critical role in ensuring that the data or information are effectively communicated. The “Output Layer” should be designed to handle a variety of

data formats and transmission protocols, allowing for seamless integration with a diverse range of systems, such as databases, data warehouses, data lakes, or real-time analytics platforms.

3.5 In-Memory Storage Layer

The “In-Memory Storage Layer” is an optional stage that comes after the “Stream Processing Layer”, providing a temporary and fast data storage solution for intermediate results or stateful computations. This layer enhances the overall performance and flexibility of the pipeline by enabling low-latency access and manipulation of the data while reducing the need for persistent storage or external databases.

Leveraging in-memory data structures and technologies, such as caching systems or distributed in-memory data grids, the “In-Memory Storage Layer” implementation should store processed data in the system's main memory (RAM) instead of traditional disk-based storage. This approach significantly improves the speed and efficiency of data retrieval and modification, allowing real-time analytics and decision-making based on the most up-to-date information.

3.6 Long-Term Storage Layer

The “Long-Term Storage Layer” is an optional stage that follows the “Stream Processing Layer”, offering a durable and persistent storage solution for processed data that needs to be retained over an extended period. This layer plays a vital role in ensuring that valuable insights derived from the data stream are securely stored, readily accessible, and available for historical analysis or future reference.

The “Long-Term Storage Layer” accommodates various storage systems, such as databases, data warehouses, or data lakes, which are designed to handle a diverse range of data formats, including structured, semi-structured, or unstructured data. This layer is responsible for efficiently storing and managing the processed data, providing mechanisms for data retrieval, querying, and analytics when needed.

The main difference between this layer and the “In-Memory Storage Layer” is that it helps store huge amounts of data for further in-depth analytics than the “In-Memory Storage Layer”.

3.7 Analytics Layer

The “Analytics Layer” is a versatile stage that can be situated after either the “In-Memory Storage Layer”, the “Long-Term Storage Layer”, or both, as well as following the “Output Layer”. This layer is responsible for extracting valuable insights, patterns, and trends from

the processed data, enabling data-driven decision-making, and enhancing the overall understanding of the underlying data.

3.8 Data Quality Layer

The “Data Quality Layer” is an independent yet essential component that permeates all other layers, ensuring the accuracy, consistency, and reliability of the data throughout the entire pipeline. By addressing data quality concerns at each stage of the pipeline, this layer plays a critical role in guaranteeing that the insights and decisions derived from the data are trustworthy and actionable.

The “Data Quality Layer” should encompass a comprehensive set of processes, techniques, and tools that are applied to each layer of the pipeline, including data validation, cleansing, transformation, deduplication, and enrichment. It also ensures data consistency by enforcing data governance policies, data lineage tracking, and metadata management.

Conclusion

This bachelor thesis is aimed to create a common data streaming architecture using scientific methods. The study began with a thorough theoretical review of the topic from definitions of what architecture is, data streaming and challenges associated with data streaming architectures in Chapter 1, to examining the history of fundamental concepts and principles in the Subchapter 2.1.

Despite initially intending to conduct a Systematic Literature Review (SLR) to identify meta-analyses on the subject, the author found no such relevant papers in the topic. Instead, a broader SLR was carried out across four scientific databases, resulting in the identification of 327 publications related to data streaming architecture. From the primary result only 2 relevant papers were found, but a further literature search in the reference lists of found papers helped to find another 5 papers, which were included in the final list of relevant literature for analysis in this thesis.

Following the objectives and questions posed for this review in Chapter 2 and through the application of induction and generalization techniques, the author was able to identify common architectural components present in the various analyzed data streaming architectures.

By synthesizing these findings, the author proposed a general data streaming architecture blueprint that encompasses all the previously described layers, providing a comprehensive and adaptable framework.

It should also be noted that since the study of possible architectures was carried out with the help of scientific methods in order to propose the most general form of architecture, there is no implementation by the author needed nor provided in this paper.

In this bachelor's thesis, all goals set were successfully achieved, from objectives to the aim of the whole thesis — proposing a general data streaming architecture. The greatest contribution of this thesis the author considers to be the proposed Data Streaming Architecture, which has parts from all the works reviewed in this thesis.

It is important to note that the suggested architecture does not serve as a step-by-step guide for implementation. Complex constructs such as data streaming architectures can vary significantly (as demonstrated in this paper) depending on the specific needs of the organization or individual implementing them. Nonetheless, this architecture primarily serves to validate the questions raised in this paper regarding the existence of common components and their generalization. As a result, the proposed architecture stands as an exemplary representation of the current knowledge in this field.

Suggestions for Future Research

While the author believes that the objectives established at the outset of this study have been fulfilled, numerous additional topics could further enrich and enhance the subject matter explored in this thesis.

This paper has thoroughly explored the primary four scientific databases, it is possible that searching additional databases may yield relevant sources that could contribute to the proposed architecture.

Future research could involve implementing the suggested architecture in real-world scenarios and comparing its flexibility, speed, and other aspects with those analyzed in this paper.

Moreover, subsequent studies could examine the specific software utilized in each of the proposed components. In addition to this, researchers could investigate the differences in implementation outcomes based on the choice of software, further enhancing the understanding of how these choices impact the overall performance and characteristics of the data streaming architecture.

Additionally, exploring alternative perspectives on data processing holds great potential for further research. As this paper primarily focuses on a novel approach to data management and processing, including a review of research papers published within the last decade, one could compare different methodologies to identify their respective advantages and drawbacks.

Another possible direction would be to examine the evolution of Data Streaming Architecture over time, investigating the implementation of architectures from various stages of the field's maturity. This approach would involve analyzing how they have evolved, the benefits they have provided then and now, and how the field adapted in response to new approaches and frameworks.

If subsequent research will share similar objectives to this study, with the aim of proposing a general architecture, it is recommended for the author to consider expanding upon the architecture presented in this work. As the current focus was rather on identifying and documenting common features, no components that were only indirectly mentioned in the analyzed articles were included in the final architecture. For instance, a “Security Layer” was not directly related to Data Streaming Architecture but could be quite important. By incorporating this layer, the proposed architecture may appear more comprehensive, resembling a complete product or blueprint that could guide implementation efforts.

Sources

Affetti, L., Tommasini, R., Margara, A., Cugola, G., & Della Valle, E. (2017). Defining the execution semantics of stream processing engines. *Journal of Big Data*, 4(1), 12. <https://doi.org/10.1186/s40537-017-0072-9>

Aggarwal, C. C. (Ed.). (2007). *Data Streams* (Vol. 31). Springer US. <https://doi.org/10.1007/978-0-387-47534-9>

Apache. (2021). *Execution Mode (Batch/Streaming)*. [//nightlies.apache.org/flink/flink-docs-release-1.14/docs/dev/datastream/execution_mode/](https://nightlies.apache.org/flink/flink-docs-release-1.14/docs/dev/datastream/execution_mode/)

Babcock, B., Babu, S., Datar, M., Motwani, R., & Widom, J. (2002). Models and issues in data stream systems. *Proceedings of the Twenty-First ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, 1–16. <https://doi.org/10.1145/543613.543615>

Carbone, P., Katsifodimos, A., Ewen, S., Markl, V., Haridi, S., & Tzoumas, K. (2015). Apache flink: Stream and batch processing in a single engine. *The Bulletin of the Technical Committee on Data Engineering*, 38(4). <http://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-198940>

Cugola, G., & Margara, A. (2012). Processing flows of information: From data stream to complex event processing. *ACM Computing Surveys*, 44(3), 15:1-15:62. <https://doi.org/10.1145/2187671.2187677>

Dunning, T., & Friedman, E. (2016). *Streaming Architecture: New Designs Using Apache Kafka and MapR Streams* (1st edition). O'Reilly Media.

Ge, S., Isah, H., Zulkernine, F., & Khan, S. (2019). A Scalable Framework for Multilevel Streaming Data Analytics using Deep Learning. *2019 IEEE 43rd Annual Computer Software and Applications Conference (COMPSAC)*, 189–194. <https://doi.org/10.1109/COMPSAC.2019.10205>

Golab, L., & Özsu, M. T. (2003). Issues in data stream management. *ACM SIGMOD Record*, 32(2), 5–14. <https://doi.org/10.1145/776985.776986>

Isah, H., Abughofa, T., Mahfuz, S., Ajerla, D., Zulkernine, F., & Khan, S. (2019). A Survey of Distributed Data Stream Processing Frameworks. *IEEE Access*, 7, 154300–154316. <https://doi.org/10.1109/ACCESS.2019.2946884>

Judalet, L. (2019, August 29). *Introduction to Event-driven Architectures With RabbitMQ*. <https://blog.theodo.com/2019/08/event-driven-architectures-rabbitmq/>

Kamburugamuve, S., & Fox, G. (2016). Survey of distributed stream processing. *Bloomington: Indiana University*, 153.

- Kitchenham, B., & Charters, S. (2007). *Guidelines for performing systematic literature reviews in software engineering*. Technical report, EBSE Technical Report EBSE-2007-01.
- Kreps, J., Narkhede, N., & Rao, J. (2011). *Kafka: A Distributed Messaging System for Log Processing*.
- Meehan, J., Aslantas, C., Zdonik, S., Tatbul, N., & Du, J. (2017). Data Ingestion for the Connected World. *CIDR*, 17, 8–11.
- Moxey, C., Edwards, M., Etzion, O., Ibrahim, M., Iyer, S., Lalanne, H., Monze, M., Peters, M., Rabinovich, Y., & Sharon, G. (2010). A conceptual model for event processing systems. *IBM Redguide Publication*.
- Psaltis, A. (2017). *Streaming Data*.
- Robins, D. (2010). Complex event processing. *Second International Workshop on Education Technology and Computer Science. Wuhan*, 1–10.
- Toshniwal, A., Taneja, S., Shukla, A., Ramasamy, K., Patel, J. M., Kulkarni, S., Jackson, J., Gade, K., Fu, M., Donham, J., Bhagat, N., Mittal, S., & Ryaboy, D. (2014). Storm@twitter. *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*, 147–156. <https://doi.org/10.1145/2588555.2595641>
- Upsolver. (2022). *An Architect's Guide to Stream Processing at Scale*. <https://www.upsolver.com/wp/stream-processing-ebook>
- Zhu, Y., & Shasha, D. (2002). StatStream: Statistical monitoring of thousands of data streams in real time. *Proceedings of the 28th International Conference on Very Large Data Bases*, 358–369.